Hands-on Labs

Afternoon Session

Day 2 - Afternoon Labs

KavinSchool

Lab 2.3: Temperature Experiment

Controlling Creativity

KavinSchool

Lab 2.3: Temperature Experiment

Goal: See how temperature affects output quality and creativity.

Tasks:

1. Pick a creative task: "Generate 5 marketing taglines for a coffee shop called Byte Brew that caters to software developers"
2. Run the same prompt at temperatures: 0.0, 0.2, 0.5, 0.7, 1.0, 1.2
3. Log all outputs side by side
4. Rate each set: creativity (1-5), relevance (1-5), coherence (1-5)
5. Find the "sweet spot" for this task

Lab 2.3: Temperature Experiment

Bonus: Try the same experiment with `top_p` values: 0.3, 0.5, 0.7, 0.9, 1.0

Time: 20 minutes | **File:** `day2/labs/lab2_3_temperature.py`

Lab 2.4: Streaming Chat Loop

Real-Time Output

KavinSchool

Lab 2.4: Streaming Chat Loop

Goal: Build a streaming chat bot that displays responses token-by-token.

Requirements:

1. Accept user input in a loop
2. Maintain conversation history
3. Stream responses to the terminal in real-time
4. Display total tokens used after each response
5. Support commands: `/clear`, `/temp 0.5` (change temperature)

Lab 2.4: Streaming Chat Loop

Stretch goals:

- Add a typing indicator while waiting for the first token
- Measure and display time-to-first-token vs total generation time

Time: 25 minutes | **File:** `day2/labs/lab2_4_streaming_chat.py`

Lab 2.5: Refactor Assistant

Build, Test, Iterate

KavinSchool

Lab 2.5: Refactor Assistant

Goal: Build a tool that takes messy Python code and returns clean, refactored code with an explanation. Iterate the prompt **3 times** based on observed failures.

Requirements:

1. Accept a Python code snippet
2. Return: refactored code + explanation of changes
3. Maintain the original behavior (no functional changes)
4. Follow PEP 8, add type hints, improve naming

Lab 2.5: Refactor Assistant

The iteration process:

1. **v1:** Write your initial prompt. Test with 3 samples. Note failures
2. **v2:** Fix the most common failure. Test again
3. **v3:** Fix the next failure. Test again
4. Document what changed and why at each iteration

Time: 30 minutes | **File:** `day2/labs/lab2_5_refactor_assistant.py`

Lab 2.5: Sample Input

```
# Messy code to refactor
def proc(d):
    r = []
    for i in d:
        if i['status'] == 'active' and i['age'] > 18:
            n = i['first'] + ' ' + i['last']
            r.append({'name': n, 'email': i['email']})
    return r
```

Expected output: Clean version with descriptive names, type hints, docstring, and explanation of every change made.

Lab 2.6: Email Drafter

Build, Test, Iterate (Without Code Prompts)

KavinSchool

Lab 2.6: Email Drafter

Goal: Build a tool that takes bullet points and returns a professional customer email. Iterate the prompt **3 times** based on observed failures.

Requirements:

1. Accept brief bullet points as input
2. Return: Professional, ready-to-send email
3. Iterate to fix tone, length, and missing call-to-actions
4. Follow business email conventions

Lab 2.6: Email Drafter

The iteration process:

1. **v1:** Write your initial prompt. Test with 3 samples. Note failures
2. **v2:** Fix the most common failure. Test again
3. **v3:** Fix the next failure. Test again
4. Document what changed and why at each iteration

Time: 30 minutes | **File:** `day2/labs/lab2_6_email_drafter.py`

Lab 2.6: Sample Input

```
- Customer: Acme Corp
- Issue: API rate limit exceeded
- Solution: Upgrade to Pro plan or optimize API calls
- Next step: Schedule a 15-min call to discuss
```

Expected output: A clear, empathetic professional email structured with a greeting, acknowledgment, solution, and clear next steps, kept under 150 words.